```
import os
import re

workspace_root = r"c:\Users\Acer\OneDrive\Desktop\SUMMER TRANNING"
output_file_path = os.path.join(workspace_root, "CARDIOGUARD_SOURCE_CODE.md")

sections = [
    {
        "title": "1. Configuration, Containerization & Build System",
        "description": "Deployment configuration, Docker containerization, package dependencies, environment manifests, and build scripts.",
        "files": [
            ("Dockerfile", "dockerfile"),
            ("render.yaml", "yaml"),
            ("package.json", "json"),
            ("requirements.txt", "text"),
            (".gitignore", "gitignore"),
            ("frontend/package.json", "json"),
            ("frontend/vite.config.ts", "typescript"),
            ("frontend/vitest.config.ts", "typescript"),
            ("frontend/tsconfig.json", "json"),
            ("frontend/tsconfig.app.json", "json"),
            ("frontend/tsconfig.node.json", "json"),
            ("frontend/vercel.json", "json"),
            ("frontend/.gitignore", "gitignore"),
            ("frontend/.oxlintrc.json", "json"),
            ("frontend/openapi.json", "json"),
        ]
    },
    {
        "title": "2. Database Schemas & SQL Migrations",
        "description": "Supabase PostgreSQL database schemas, table definitions, indexes, Row Level Security (RLS) policies, and trigger procedures.",
        "files": [
            ("frontend/supabase/migrations/master_schema.sql", "sql"),
            ("frontend/supabase/migrations/01_create_tables.sql", "sql"),
            ("frontend/supabase/migrations/02_indexes.sql", "sql"),
            ("frontend/supabase/migrations/03_rls.sql", "sql"),
            ("frontend/supabase/migrations/04_triggers.sql", "sql"),
            ("frontend/supabase/migrations/05_add_patient_role.sql", "sql"),
            ("frontend/supabase/migrations/06_fix_rls.sql", "sql"),
            ("frontend/supabase_schema.sql", "sql"),
```

```
    ]
  },
  {
    "title": "3. Backend API Server & Business Logic",
    "description": "Node.js Express application entry point, routing, middleware (auth, rate limiting, error handling), validation schemas, and domain services.",
    "files": [
      ("backend/server.js", "javascript"),
      ("backend/db.js", "javascript"),
      ("backend/jsonDb.js", "javascript"),
      ("backend/config/env.js", "javascript"),
      ("backend/middleware/authMiddleware.js", "javascript"),
      ("backend/middleware/errorMiddleware.js", "javascript"),
      ("backend/middleware/rateLimiter.js", "javascript"),
      ("backend/routes/predictionRoutes.js", "javascript"),
      ("backend/routes/statsRoutes.js", "javascript"),
      ("backend/services/predictionService.js", "javascript"),
      ("backend/services/benchmarkService.js", "javascript"),
      ("backend/services/recommendationService.js", "javascript"),
      ("backend/services/statsService.js", "javascript"),
      ("backend/utils/logger.js", "javascript"),
      ("backend/utils/python.js", "javascript"),
      ("backend/validators/inputValidator.js", "javascript"),
    ]
  },
  {
    "title": "4. Machine Learning Engine & Data Science Pipelines",
    "description": "Python model inference microservices, model training pipelines, EDA analysis, model benchmarking, and worker processes.",
    "files": [
      ("backend/ml_service.py", "python"),
      ("backend/benchmark_service.py", "python"),
      ("backend/benchmark.py", "python"),
      ("backend/predict_worker.py", "python"),
      ("notebooks/01_EDA.py", "python"),
      ("notebooks/02_ML_Pipeline.py", "python"),
      ("notebooks/train_and_evaluate.py", "python"),
    ]
  },
  {
    "title": "5. Frontend Application Layer",
```

```
        "description": "React 18 + TypeScript SPA components, pages, custom hooks, context providers, layout, styling, and Supabase client integration.",
        "files": [
            ("frontend/index.html", "html"),
            ("frontend/public/favicon.svg", "xml"),
            ("frontend/public/icons.svg", "xml"),
            ("frontend/src/main.tsx", "tsx"),
            ("frontend/src/App.tsx", "tsx"),
            ("frontend/src/App.css", "css"),
            ("frontend/src/index.css", "css"),
            ("frontend/src/types/index.ts", "typescript"),
            ("frontend/src/supabase/index.ts", "typescript"),
            ("frontend/src/supabase/types.ts", "typescript"),
            ("frontend/src/context/AuthContext.tsx", "tsx"),
            ("frontend/src/context/NotificationContext.tsx", "tsx"),
            ("frontend/src/context/ThemeContext.tsx", "tsx"),
            ("frontend/src/hooks/useStats.ts", "typescript"),
            ("frontend/src/components/Logo.tsx", "tsx"),
            ("frontend/src/components/Skeleton.tsx", "tsx"),
            ("frontend/src/components/EmptyState.tsx", "tsx"),
            ("frontend/src/components/RiskGauge.tsx", "tsx"),
            ("frontend/src/components/Layout.tsx", "tsx"),
            ("frontend/src/components/ReportTemplate.tsx", "tsx"),
            ("frontend/src/components/BenchmarkAnalyticsDashboard.tsx", "tsx"),
            ("frontend/src/pages/LandingPage.tsx", "tsx"),
            ("frontend/src/pages/LoginPage.tsx", "tsx"),
            ("frontend/src/pages/RegisterPage.tsx", "tsx"),
            ("frontend/src/pages/ForgotPasswordPage.tsx", "tsx"),
            ("frontend/src/pages/DashboardPage.tsx", "tsx"),
            ("frontend/src/pages/PredictPage.tsx", "tsx"),
            ("frontend/src/pages/HistoryPage.tsx", "tsx"),
            ("frontend/src/pages/InsightsPage.tsx", "tsx"),
            ("frontend/src/pages/AdminDashboard.tsx", "tsx"),
            ("frontend/src/setupTests.ts", "typescript"),
            ("frontend/src/App.test.tsx", "tsx"),
        ]
    }
]


def sanitize_content(content):
    # Mask any potential JWT tokens or explicit secret strings if present
```

```
    content = re.sub(r'eyJ[A-Za-z0-9-_=]+\.[A-Za-z0-9-_=]+\.[A-Za-z0-9-_.+/=]+',
'[REDACTED_JWT_TOKEN]', content)
    content = re.sub(r'sbp_[a-zA-Z0-9]{20,}', '[REDACTED_SUPABASE_SERVICE_KEY]', content)
    return content

# Calculate summary stats
file_metadata = []
total_lines_all = 0
total_size_all = 0

for section in sections:
    for rel_path, lang in section["files"]:
        full_path = os.path.join(workspace_root, rel_path.replace('/', os.sep))
        if os.path.exists(full_path):
            size_bytes = os.path.getsize(full_path)
            with open(full_path, 'r', encoding='utf-8', errors='ignore') as f:
                lines = f.readlines()
                line_count = len(lines)
            total_lines_all += line_count
            total_size_all += size_bytes
            file_metadata.append({
                "rel_path": rel_path,
                "lang": lang,
                "section": section["title"],
                "lines": line_count,
                "size_bytes": size_bytes
            })
        else:
            print(f"WARNING: File not found: {rel_path}")

print(f"Total files: {len(file_metadata)}")
print(f"Total lines: {total_lines_all}")
print(f"Total size: {total_size_all} bytes ({total_size_all/1024:.2f} KB)")

# Generate Markdown Document
with open(output_file_path, 'w', encoding='utf-8') as out:
    out.write("# CardioGuard AI — Complete Source Code Reference Document\n\n")
    out.write("> **System Name:** CardioGuard AI — Explainable Heart Disease Risk Predictor  \n")
    out.write("> **Repository Root:** `c:\\Users\\Acer\\OneDrive\\Desktop\\SUMMER TRANNING`  \n")
    out.write(f"> **Total Source Files:** {len(file_metadata)}  \n")
    out.write(f"> **Total Lines of Code:** {total_lines_all:,} lines  \n")
    out.write(f"> **Generated Date:** 2026-08-18  \n\n")
```

```
    out.write("---\n\n")
    out.write("## Executive Overview & Audit Compliance\n\n")
    out.write("This document provides the **complete, exact, and un-truncated source code** for the CardioGuard AI application. ")
    out.write("It encompasses all functional components across the system stack, including Frontend UI, Backend Microservices, Machine Learning & Data Science Pipelines, Database Schema Migrations, and Deployment Infrastructure.\n\n")

    out.write("### Exclusion & Security Policy Compliance\n")
    out.write("Per project auditing guidelines, the following sensitive and auto-generated non-source artifacts have been excluded or sanitized:\n")
    out.write("- **Secrets & Environment Credentials:** `.env` and `.env.local` files containing live API keys, JWT secrets, or DB passwords are excluded. Secret strings in defaults have been replaced with `[REDACTED_SECRET]` placeholders.\n")
    out.write("- **Dependencies & Caches:** `node_modules/`, Python `.venv/` / `venv/`, and `.pytest_cache/` directories are excluded.\n")
    out.write("- **Binary & Generated Artifacts:** Compiled model pickles (`models/*.pkl`), binary graphics (`assets/*.png`, `.pdf`), runtime database dumps (`db.json`), lockfiles (`package-lock.json`), and build outputs (`dist/`, `.vercel/`) are excluded.\n\n")

    out.write("---\n\n")
    out.write("## Master Directory & Table of Contents\n\n")
    out.write("| # | Section | File Path | Language | Lines | Size (KB) |\n")
    out.write("|---|---|---|---|---|---|\n")

    idx = 1
    for meta in file_metadata:
        anchor = meta['rel_path'].lower().replace('/', '').replace('.', '').replace('_', '')
        size_kb = meta['size_bytes'] / 1024.0
        out.write(f"| {idx} | {meta['section'].split('.')[1].strip() if '.' in meta['section'] else meta['section']} | [`{meta['rel_path']}`](#file-{anchor}) | `{meta['lang']}` | {meta['lines']} | {size_kb:.2f} KB |\n")
        idx += 1

    out.write(f"| **Total** | **All 5 Core Modules** | **{len(file_metadata)} Files** | — | **{total_lines_all:,}** | **{total_size_all/1024:.2f} KB** |\n\n")
    out.write("---\n\n")

    # Process each section
    for section in sections:
        out.write(f"## {section['title']}\n\n")
        out.write(f"*{section['description']}*\n\n")

        for rel_path, lang in section["files"]:
            full_path = os.path.join(workspace_root, rel_path.replace('/', os.sep))
```

```
        anchor = rel_path.lower().replace('/', '').replace('.', '').replace('_', '')

        if not os.path.exists(full_path):
            continue

        size_bytes = os.path.getsize(full_path)
        with open(full_path, 'r', encoding='utf-8', errors='ignore') as f:
            raw_code = f.read()

        lines_count = len(raw_code.splitlines())
        clean_code = sanitize_content(raw_code)

        out.write(f"<a id=\"file-{anchor}\"></a>\n")
        out.write(f"### File: `{rel_path}`\n\n")
        out.write(f"> **Path:** `{rel_path}`  \n")
        out.write(f"> **Language:** `{lang}` | **Lines:** `{lines_count}` | **Size:** `{size_bytes / 1024.0:.2f} KB`\n\n")

        out.write(f"```{lang}\n")
        out.write(clean_code)
        if not clean_code.endswith('\n'):
            out.write('\n')
        out.write("```\n\n")
        out.write("---\n\n")

print("Document generated successfully at:", output_file_path)
```